PORTALDOT HACKER HOUSE 2026 · MODULAR CLOUD & MIDDLEWARE

Make a sponsored write on Portaldot. You sign. A relayer pays.

Plaque is a reference implementation of a sponsored (gasless) write: a user with a zero balance signs once, a relayer verifies and pays, and the chain records who claimed. The handle registry is the example write. It is live on the V3 testnet — the proof claim was signed by an account holding 0 tPOT.

CONTRACT

0x5432e6b89dc3042d
47f2be98e971cad9
dabab685

LIVE CLAIM

block **167,209**
claimant balance **0 wei**
sponsor paid **3.87 tPOT**

VERIFY IT

npm run verify:live
10 checks, all passing

THE PROBLEM

A brand-new account holds zero, so it cannot write anything.

On Portaldot a new account starts at 0 tPOT. It cannot send a transaction, claim a name, register for a program or leave any trace on chain.

Testnet POT is handed out by hand through a Discord support ticket. So the first experience of the ecosystem is: wait for a human.

And every dApp that wants to grow has the same wall: it must ask people to buy gas before they can use it. The reusable answer is a sponsored write; Plaque makes it a module another dApp can copy.

THE CHAIN ALREADY SHIPS THE FIX

`MetaTx` — fee payment can be delegated, so a signature can be submitted by somebody else.

`VerifySignature` — the runtime can check a signature natively.

`pallet-revive` — an EVM, so ordinary Solidity works.

Plaque uses an ordinary sponsor transaction today; MetaTx is roadmap.

THE SOLUTION

Three steps. The relayer pays.

01

You sign

Pick a handle. Your wallet signs one EIP-191 message. No transaction, no gas, no account funding.

02

A sponsor pays

A relayer — or your own wallet — submits the signed claim and pays the chain's fee. The sponsor is recorded in the event.

03

The contract binds it

It recovers the signer itself and writes the handle to that address. Anyone can read it back from the public RPC.

The digest binds the handle, the claimant, the chain id and the contract — so a signed claim cannot be replayed onto another name, another address, another chain or another contract.

```
keccak256(abi.encode(bytes32(1), handle, who, 420420777,
contract)) → personal_sign
```

LIVE PROOF — NOT A MOCK

One command re-checks the whole thing.

```
$ npm run verify:live
Plaque live verification - block 171462

  PASS  deployed code equals the committed build  302 bytes
  PASS  ownerOf("unify") is the zero-balance claimant
  PASS  the claimant still holds 0 wei
  PASS  the Claimed event is on chain  block 167209
  PASS  the event names the claimant as `who`
  PASS  the event names the sponsor that paid
  PASS  a valid signature is accepted (eth_call)
  PASS  a tampered signature is refused
  PASS  a signature for another address is refused
  PASS  a zero address is refused

ALL CHECKS PASSED
```

THE CLAIM

tx 0xa654bd9ce55b389c
072b00d0c4af55ab06
7d76acc422d5636b0c
62863786edf2

signed by **0xd35404…15Fe6**
balance before / after: **0 / 0 wei**
paid by **0x064e2C…4fD5**

The deployed bytecode is byte-for-byte the committed source. The signature checks run against live state, so they prove the contract’s verification on the real runtime.

BUILT FOR THIS CHAIN, MEASURED

The runtime taught us two things. We built to both.

1 · NO BLOCK VALUES

This EVM reverts on the BLOCK-family opcodes — `CHAINID`, `TIMESTAMP`, `NUMBER`. Two probe contracts in the repo prove it with reverted transactions.

So no deployed build reads a block value: the chain id is a constant, and the digest binds it instead.

2 · DEPOSITS, NOT JUST GAS

signature check + compute	`0.028 tPOT`
storage deposit per new entry	`3.84 tPOT`
code deposit (302 bytes)	`36.7 tPOT`

786 bytes cost 74.5 tPOT to deploy, which is why the live build is `PlaqueLite.sol`: 302 bytes, same digest and storage slot; the lite event is the three-address form, differential-tested against the readable build.

TRUST MODEL

The contract trusts nothing but the signature.

No custody

No escrow, no bond, no arbitration, no mint. A claim carries value 0; the lite fallback is payable but has no withdraw, so do not send tPOT to the contract. The only value that moves is the chain’s own deposit, paid by the sponsor to the runtime.

An untrusted sponsor

It can refuse to pay, reorder claims or go offline. It cannot forge a claim, and it cannot bind a name to somebody else’s address.

A dropped claim is recoverable

The claim is a signed message, not a secret. Anybody can resubmit it — including the claimant with their own gas.

A name stays bound in contract state, and the registry is readable by any app from the public RPC.

Known limit, stated openly: a free name can be squatted by binding it to an address the squatter controls. Here that costs ≈3.87 tPOT per name — expensive enough to deter, and a claim-then-confirm grace period would remove it.

WHO IT IS FOR

A reusable rail, with a registry as its first consumer.

Users. The ~168 hackers in this event, ecosystem newcomers, and anyone who wants a name on chain before they have funds.

Builders. Any Portaldot dApp can reuse the sponsored-write path instead of asking users to buy gas first. `docs/INTEGRATE.md` is the under-10-minute path: signature verification, policy caps, nonce queue, config and a reader.

The ecosystem. The repository documents the chain's real cost and opcode constraints with transactions — facts every builder here will hit.

THE BUSINESS

The sponsor is the customer: a hosted relayer with a per-developer budget, settled in the chain's native `Assets` pallet so sponsorship itself is on chain.

Unit economics are measured, not guessed: 3.87 tPOT per sponsored write on this testnet, dominated by the storage deposit.

The registry is the reference consumer; the reusable artifact is the sponsored-write rail. The integration guide is tested against a local chain with refusals and positive controls.

WHERE IT GOES NEXT

Roadmap, in the order the risk disappears.

STEP	WHY	STATE
Claim-then-confirm grace period	removes name griefing entirely	next
Hosted sponsor + on-chain budget	a real service a dApp can point at	designed
Release; signed transfers	names become movable property	release is in the readable build; transfers roadmap
Public handle indexer	any Portaldot app resolves names from the RPC	planned
Mainnet, cost model re-measured	the same contract, real deposits	after the testnet

The hard part is done and on chain: a signature-only claim from a zero-balance account, verified byte-for-byte, readable by anyone.

PLAQUE

A sponsored write on Portaldot, without buying gas first.

app https://plaque-oxunify.vercel.app
repo https://github.com/unifyWeb3/plaque
verify npm run verify:live
tests npm test — 41 / 41

contract 0x5432e6b8…b685
deploy tx 0xcee4d1db…3b3a · block 167,044
claim tx 0xa654bd9c…edf2 · block 167,209
chain Portaldot V3 testnet · 420420777

Testnet only. No audited code, no real value, no mainnet deployment. Built by unify for the Portaldot Hacker House 2026.